

UITS

UNIVERSITY OF INFORMATION
TECHNOLOGY AND SCIENCES

Project Report

Course Title: Software Engineering and System Analysis Lab

Course Code: CSE 356

Submitted to:

Dhrubo Barua
Lecturer
Department of CSE, UITS

Md Foysal
Lecturer
Department of CSE, UITS

Submitted by:

- ✧ **Name:** Faria Khanam Jerin
ID: 0432320005101002
- ✧ **Name:** Jahid Hassan Sowrov
ID: 0432320005101010
- ✧ **Name:** Zearin Akhter
ID: 0432320005101014
- ✧ **Name:** Fatema Akter Lamia
ID: 0432320005101015
Department: CSE
Batch: 54th
Semester: Spring 2026
Section: 6A

Date of Submission: 20th May, 2026

Acknowledgement

First of all, we express our deepest gratitude to Almighty Allah for giving us the strength, patience, and ability to complete this project successfully. We are also thankful to our respected teachers, Dhrubo Barua and Md Foysal, for their continuous guidance, support, and valuable suggestions throughout the project development and documentation process.

We would also like to thank the Department of Computer Science and Engineering, UITS, for giving us the opportunity to work on a practical system analysis and software engineering project. This project helped us understand requirement analysis, software design, database development, user interface design, testing, and project documentation more clearly.

Finally, we are grateful to our family members and friends for their encouragement and moral support during the development of this project.

Dedication

We dedicate this project to our parents, teachers, and all responsible citizens who believe in justice, honesty, and social accountability. This project is also dedicated to people who face difficulties when reporting corruption but still want to contribute to a transparent and fair society.

Abstract

The Anti-Corruption Reporting System (ACRS) is a secure web-based platform designed to help citizens report corruption and crime-related incidents through an organized digital system. Traditional reporting methods are often slow, paper-based, difficult to track, and may expose the identity of whistle-blowers. ACRS addresses these issues by offering secure authentication, anonymous reporting, evidence upload, unique tracking ID generation, complaint status tracking, admin case management, notifications, and feedback collection.

The system is developed using HTML, CSS, JavaScript, PHP, MySQL, and XAMPP. It contains two main modules: the Citizen Portal and the Admin/Authority Dashboard. Citizens can submit complaints with optional evidence and track complaint progress using a unique tracking ID. Admins can review complaints, assign departments, update case statuses, and view user feedback.

This report presents the complete analysis, design, implementation, testing, and evaluation of the ACRS project. It includes the Software Requirement Specification, DFD Level 0, DFD Level 1, Use Case Diagram, ER Diagram, Sequence Diagram, State Diagram, Gantt Chart, system screenshots, test cases, cost analysis, SWOT analysis, and future improvement plan.

Table of Contents

Acknowledgement	ii
Dedication	iii
Abstract	iv
Chapter 1: Introduction	8
Chapter 2: Software Requirement Specification	12
Chapter 3: System Analysis	16
Chapter 4: System Design	18
Chapter 5: Implementation and User Interfaces	25
Chapter 6: System Testing	41
Chapter 7: Discussion, Conclusion and Future Work	42
References	43
Core Implementation Code	44

List of Figures

Figure 1: Gantt Chart of Anti-Corruption Reporting System

Figure 2: DFD Level 0 of Anti-Corruption Reporting System

Figure 3: DFD Level 1 of Anti-Corruption Reporting System

Figure 4: Use Case Diagram of ACRS

Figure 5: Entity Relationship Diagram of ACRS

Figure 6: Full Sequence Diagram of ACRS

Figure 7: State Diagram of Complaint Processing

Figure 8: Home Page of ACRS

Figure 9: Login Page of ACRS

Figure 10: Signup Page of ACRS

Figure 11: Signup Success Message

Figure 12: User Manual Page in Bengali

Figure 13: User Manual Status Tracking Instructions

Figure 14: Submit Report Page

Figure 15: My Submitted Complaints Page

Figure 16: Admin Control Center with Reports

Figure 17: Admin Status Update Example

Figure 18: Updated Complaint List with Notification

Figure 19: Track Complaint and Feedback Form

Figure 20: Feedback Submitted View

List of Tables

Table 1: Project Stakeholders

Table 2: Functional Requirements

Table 3: Non-Functional Requirements

Table 4: Database Tables

Table 5: SWOT Analysis

Table 6: Cost Analysis

Table 7: Source Code File Structure

Chapter 1

Introduction

1.1 Project Overview

The Anti-Corruption Reporting System (ACRS) is a secure web-based platform that allows citizens to report corruption and crime-related incidents through an online portal. The system is designed to reduce dependency on paper-based reporting and to make complaint management more transparent, accountable, and easy to access. It provides a structured connection between citizens and authorities, where citizens can submit complaints, attach evidence, track the status of their complaints, and give feedback after a case is resolved.

1.2 Background

In many traditional systems, citizens must physically visit an office to report corruption or misconduct. This process is time-consuming and may discourage people from reporting due to fear, lack of privacy, or inconvenience. Paper-based reports can also be misplaced or delayed. A digital reporting system can solve many of these problems by storing records in a database, generating a tracking ID, and allowing administrators to update case progress in real time.

1.3 Problem Statement

Many citizens face major barriers when reporting corruption. These barriers include lack of anonymity, fear of retaliation, inefficient paperwork, lack of tracking, delayed administrative response, and limited access for rural or physically restricted users. Once a report is filed manually, the whistle-blower often has no clear way to know whether the report is being reviewed, forwarded, or resolved. This creates a transparency gap and reduces public trust.

1.4 Project Purpose

The purpose of ACRS is to develop a web-based complaint reporting platform that improves transparency, security, and efficiency. The system provides separate access for citizens and administrators. Citizens can submit reports securely and track them using a unique complaint ID. Administrators can manage submitted reports, assign departments, update statuses, and view feedback.

1.5 Objectives

The main objectives of the project are to create a centralized corruption reporting system, protect user identity, reduce paperwork, provide real-time tracking, improve administrative accountability, and build public trust through transparent status updates.

1.6 Scope of the Project

The scope of the project includes citizen registration, login, complaint submission, anonymous reporting, evidence upload, complaint tracking, admin authentication, complaint management, department assignment, status update, notification generation, and feedback collection. The system is focused on web-based operation and is designed for use through common browsers such as Chrome and Firefox.

1.7 Project Integrity

The project follows the principle of administrative accountability. Every complaint is stored with structured information in the database, and each complaint receives a unique tracking ID. This ensures that reports cannot be ignored easily and that users can observe the progress of their submitted complaints.

1.8 Expected Outcome

After successful implementation, the system will allow citizens to report corruption more confidently and allow authorities to manage reports efficiently. It will also support digital documentation, better case tracking, and improved communication between the public and administrative authority.

1.9 Stakeholders

Stakeholder	Role in the System
Citizen/User	Submits complaints, uploads evidence, tracks status, and provides feedback.
Admin/Authority	Reviews complaints, assigns departments, updates statuses, and manages cases.
Developer	Designs, implements, tests, and maintains the software system.
Government/Organization	Uses the platform to improve transparency and accountability.
Public Community	Benefits from improved corruption reporting and public updates.

1.10 Project Schedule

Gantt Chart

The project schedule was prepared in a Gantt chart format. It includes the major development phases from planning and design to testing, auditing, and debugging.

	A	B	C	D	E	F	G	H	I	J
1	Gantt chart of Project: Anti-Corruption Reporting System									
2	Project Name: Anti-Corruption Reporting System				Today's Date: 19/05/2026					
3	Start Date: 05 February 2026									
4	■ COMPLETED ■ IN PROGRESS				■ PENDING		■ ★New Feature			
5										
6	WBS	Tasks	Task Lead	Start	End	Duration (Days)	% Done	Working Days	Days Done	Days Left
7	1	Phase 1: Design (HTML/CSS/JS)		05/02/26	28/02/26	24	100%	24	24	0
8	1.1	Citizen portal landing page & navigation		05/02/26	10/02/26	6	100%	6	6	0
9	1.2	Complaint submission form layout		11/02/26	16/02/26	6	100%	6	6	0
10	1.3	Official / Investigator dashboard		17/02/26	22/02/26	6	100%	6	6	0
11	1.4	Complaint tracking status & timeline tracker		23/02/26	28/02/26	6	100%	6	6	0
12	2	Phase 2: Core PHP Architecture		1/3/2026	15/3/26	15	100%	15	15	0
13	2.1	PHP environment configuration		1/3/2026	4/3/2026	4	100%	4	4	0
14	2.2	Modular PHP include Layouts		5/3/2026	08/03/26	4	100%	4	4	0
15	2.3	Dynamic request routing		09/03/26	11/03/26	3	100%	3	3	0
16	2.4	Tracking ID generation controller		12/03/26	13/03/26	2	100%	2	2	0
17	2.5	Role-based session handler		14/03/26	15/3/26	2	100%	2	2	0
18	3	Phase 3: Dtabase Creation & Modeling		16/3/26	28/03/26	13	100%	13	13	0
19	3.1	Design Relational Schema		16/3/26	18/03/26	3	100%	3	3	0
20	3.2	DB creation via phpMyAdmin		19/03/26	22/03/26	4	100%	4	4	0
21	3.3	Establishing key constraints & cascading action		23/03/26	25/03/26	3	100%	3	3	0
22	3.4	Database integrity testing & validation		26/03/26	28/03/26	3	100%	3	3	0
23	4	Phase 4: PHP: Frontend ↔ Database		28/03/26	09/04/26	13	100%	13	13	0
24	4.1	PHP CRUD – submitting complaints		28/03/26	31/03/26	4	100%	4	4	0
25	4.2	PHP CRUD – Updating investigation statuses		1/4/2026	03/04/26	3	100%	3	3	0
26	4.3	Connect frontend forms to target directories		04/04/26	06/04/26	3	100%	3	3	0
27	4.4	Dynamic DB-driven statistics generation		07/04/26	09/04/26	3	100%	3	3	0
28	5	Phase 5: Secure Document Handling ★		10/04/26	24/04/26	15	0%	15	15	15
29	5.1	Secure upload handler & backend validation		10/04/26	13/04/26	4	100%	4	4	4
30	5.2	Rating & feedback indicators		14/04/26	17/04/26	4	100%	4	4	4
31	5.3	Automated cryptographic engine for files		18/04/26	21/04/26	4	100%	4	4	4
32	5.4	Evidence access control checks		22/04/26	24/04/26	3	100%	3	3	3
33	6	Phase 6: Testing, Audit & Debugging		25/04/26	15/05/26	21	100%	21	0	21
34	6.1	Functional validation testing		25/04/26	29/05/26	5	100%	5	5	5
35	6.2	Security audit & SQL Injection patching		30/04/26	04/05/26	5	100%	5	5	5
36	6.3	Performance & load testing		05/05/26	09/05/26	5	100%	5	5	5
37	6.4	Final observation & optimization		10/5/2026	15/05/26	6	100%	6	6	6
38										

Figure 1: Gantt Chart of Anti-Corruption Reporting System

Chapter 2

Software Requirement Specification

2.1 Functional Requirements

Req. ID	Requirement	Description
FR-01	User Registration	The system shall allow citizens to create an account using name, email, role, NID number, and password.
FR-02	Secure Login	The system shall validate user and admin credentials using the MySQL database.
FR-03	Submit Complaint	The system shall allow users to submit complaint descriptions and select complaint categories.
FR-04	Upload Evidence	The system shall allow users to upload image or document evidence where applicable.
FR-05	Anonymous Reporting	The system shall allow users to submit complaints anonymously.
FR-06	Generate Tracking ID	The system shall generate a unique complaint ID for each submitted report.
FR-07	Track Status	The system shall allow users to track complaint status using the tracking ID.
FR-08	Admin Case Management	The system shall allow admins to view, categorize, assign, and update complaints.
FR-09	Notifications	The system shall show updates when complaint status changes.
FR-10	Feedback	The system shall allow users to provide rating and comments after complaint resolution.

2.2 Non-Functional Requirements

Req. ID	Category	Description
NFR-01	Security	User identities and complaint details must be protected.
NFR-02	Availability	The system should be available 24/7 where server support is active.
NFR-03	Usability	The interface should be simple and understandable for citizens and admins.
NFR-04	Performance	Pages should load quickly under normal use.
NFR-05	Reliability	Complaint data must be stored accurately in the database.
NFR-06	Maintainability	Code should be organized for future updates and debugging.
NFR-07	Scalability	The system should support future expansion such as mobile app or AI-based filtering.
NFR-08	Transparency	Every complaint should remain traceable through a tracking ID.

2.3 Purpose

The purpose of this SRS section is to describe the functional and non-functional requirements of the Anti-Corruption Reporting System. The system is intended to be a secure web-based platform where citizens can submit corruption reports and authorities can manage them with accountability.

2.4 Document Convention

The report uses formal academic formatting. Main headings are written in bold, body text is written in Times New Roman, and requirement tables are organized clearly for easy reading.

2.5 Intended Audience

The intended audience includes supervisors, developers, testers, administrators, and academic evaluators. Supervisors use the document for project validation, developers use it for implementation logic, testers use it for verification, and administrators use it to understand system workflow.

2.6 Product Perspective

ACRS is a self-contained web application. It replaces inefficient manual complaint reporting with a centralized digital platform. The application connects the citizen portal, PHP backend, MySQL database, and admin dashboard.

2.7 Product Functions

The main product functions are secure authentication, anonymous reporting, evidence management, unique tracking ID generation, real-time tracking, admin dashboard control, feedback collection, and public update management.

2.8 User Classes and Characteristics

There are two main classes of users. Citizens use the system to report and track complaints. Admins use the system to review complaints, assign departments, update statuses, and monitor feedback.

2.9 Operating Environment

The system operates in a local server environment using XAMPP. Apache is used as the web server, MySQL is used as the database, PHP is used for backend logic, and the application runs in standard web browsers.

2.10 Design and Implementation Constraints

The project is constrained by the use of PHP, MySQL, HTML, CSS, JavaScript, and XAMPP. Security is an important constraint because the system handles sensitive complaint data and user identities.

2.11 Assumptions and Dependencies

The system assumes that users have access to the internet and a modern web browser. It also assumes that the server remains available, and the database connection works properly.

Chapter 3

System Analysis

3.1 Feasibility Study

The project is technically, economically, and operationally feasible. It uses common web technologies, requires a low development cost, and solves a real social and administrative problem.

3.2 Technical Feasibility

The system is developed with HTML, CSS, JavaScript, PHP, MySQL, and XAMPP. These tools are widely available, easy to configure, and suitable for a student-level web-based system.

3.3 Economic Feasibility

The estimated development cost ranges from BDT 27,000 to BDT 65,000. Since the project uses mostly free and open-source tools, the cost remains manageable.

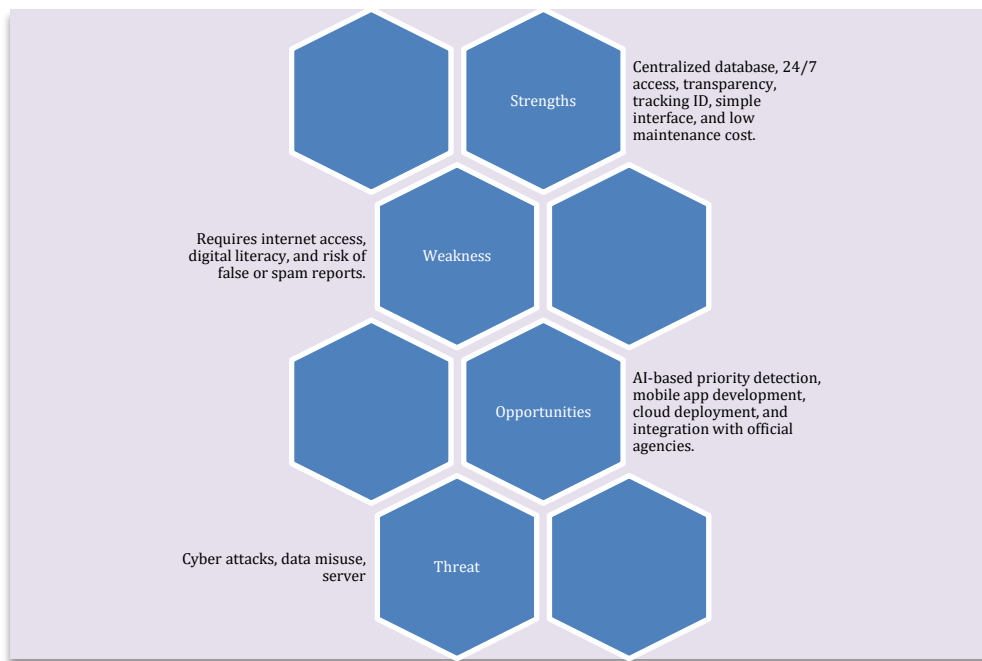
3.4 Operational Feasibility

The system is simple enough for citizens and admins to use. Citizens can submit complaints through a form and admins can update statuses from a dashboard.

3.5 Requirement Analysis

The major requirements were identified from the SRS, proposal, diagrams, and implemented system interface. The requirements focus on secure access, complaint submission, tracking, admin management, and feedback.

3.6 SWOT Analysis



3.7 Cost Analysis

Section	Minimum Cost (BDT)	Maximum Cost (BDT)
Development	15,000	40,000
Hosting and Domain	4,000	8,000
Tools	0	2,000
Maintenance	8,000	15,000
Total	27,000	65,000

Chapter 4

System Design and Diagrams

4.1 Data Flow Diagram Level 0

DFD Level 0 shows the overall interaction between external entities and the Anti-Corruption Reporting System. The Citizen sends login credentials, complaint details, evidence files, feedback, and ratings to the system. The system returns complaint status, public updates, and unique complaint IDs. The Admin/Authority receives complaint reports, evidence files, feedback reports, case categories, and case updates, while also providing admin credentials and public announcements.

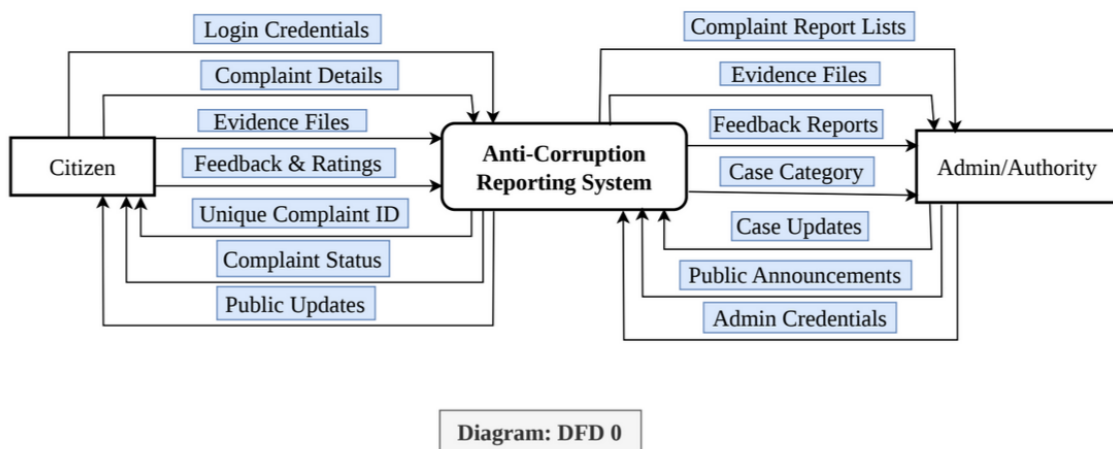


Figure 2: DFD Level 0 of Anti-Corruption Reporting System

4.2 Data Flow Diagram Level 1

DFD Level 1 breaks the system into internal processes including Manage Authentication, Process Complaint, Track Complaint, Manage Cases, Notification System, and Process Feedback. It also shows the data stores used by the system: Users DB, Complaints DB, Feedback DB, and Notifications DB.

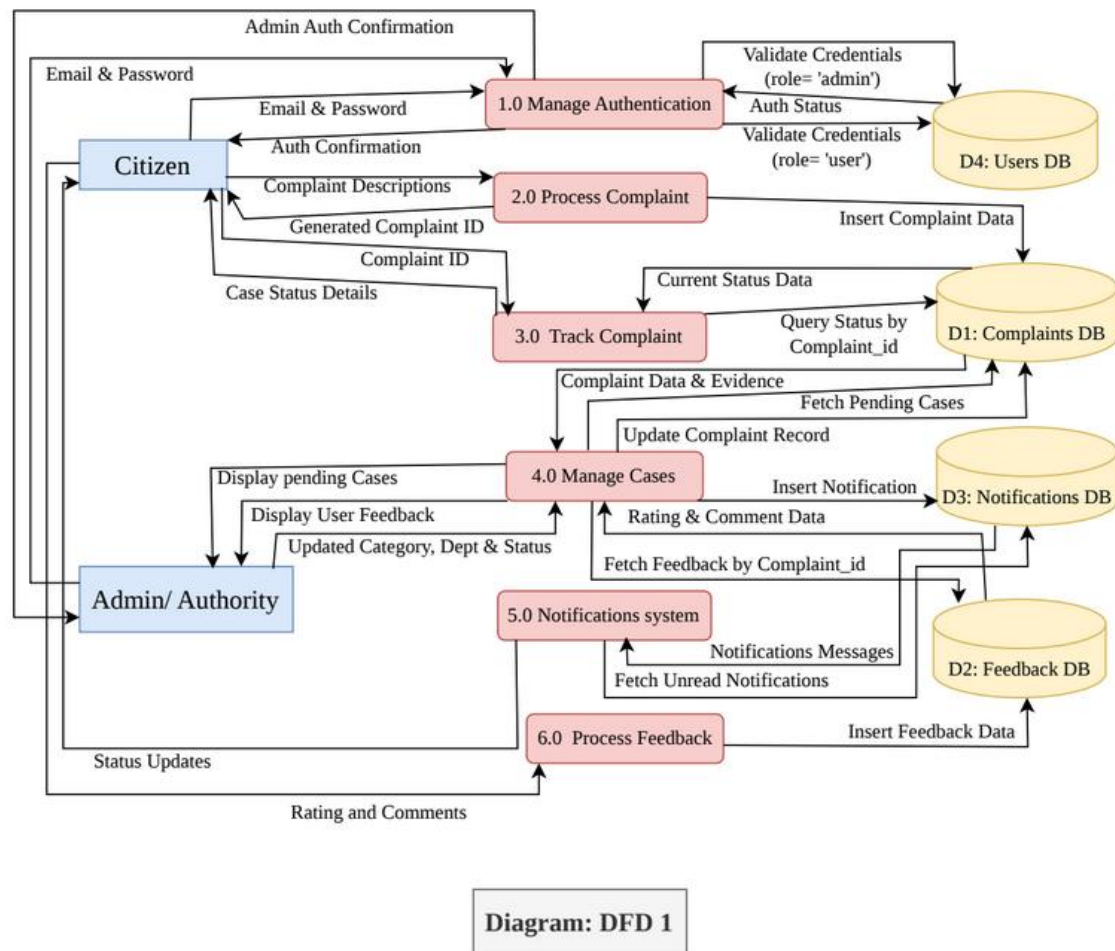


Figure 3: DFD Level 1 of Anti-Corruption Reporting System

4.3 Use Case Diagram

The Use Case Diagram shows the main interactions between the Citizen, Administrator, and the ACRS system. Citizens can register, login, file complaints, upload evidence, submit anonymously, track complaints, provide feedback, and view public updates. Administrators can login, manage complaints, update case status, post public updates, and view feedback.

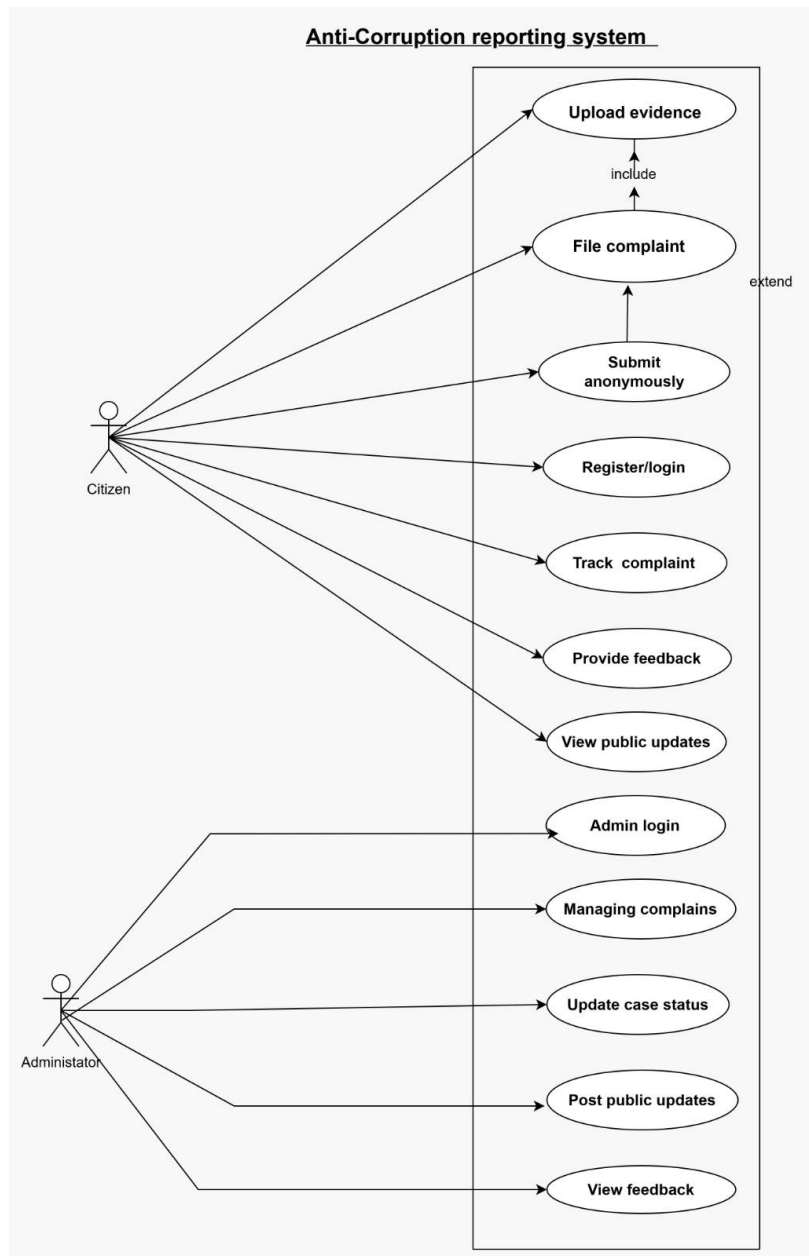


Figure 4: Use Case Diagram of ACRS

4.4 Entity Relationship Diagram

The ER Diagram represents the database structure of the system. The main entities are Users, Complaints, Notifications, and Feedback. One user can submit many complaints and receive many notifications. A complaint can generate notifications and may receive feedback after resolution.

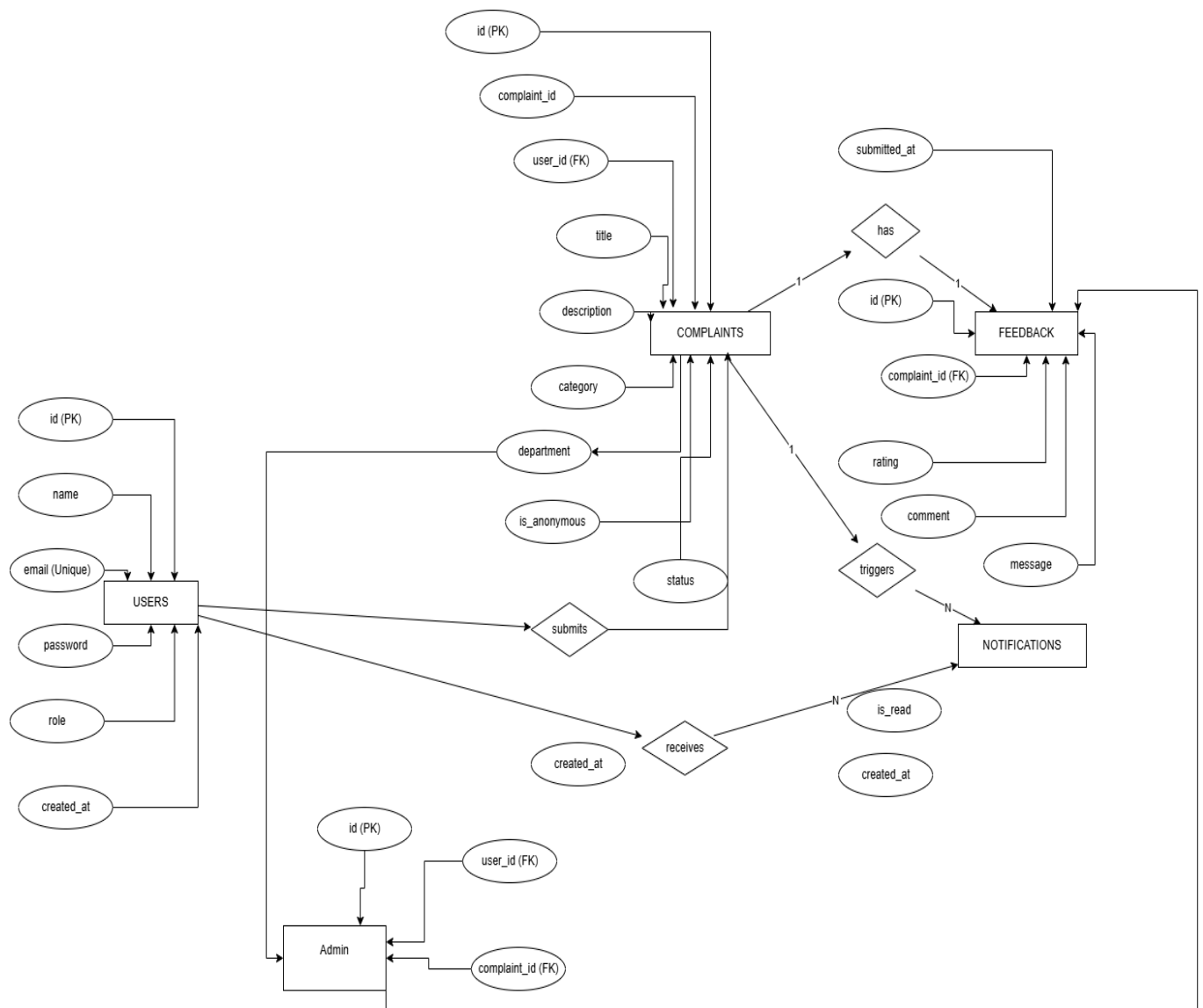


Figure 5: Entity Relationship Diagram of ACRS

4.5 Sequence Diagram

The Sequence Diagram represents the time-based interaction among Citizen, Web Interface, PHP Controller, MySQL Database, and Admin/Authority. It shows registration, login, complaint submission, tracking, admin status update, notification generation, and feedback submission.

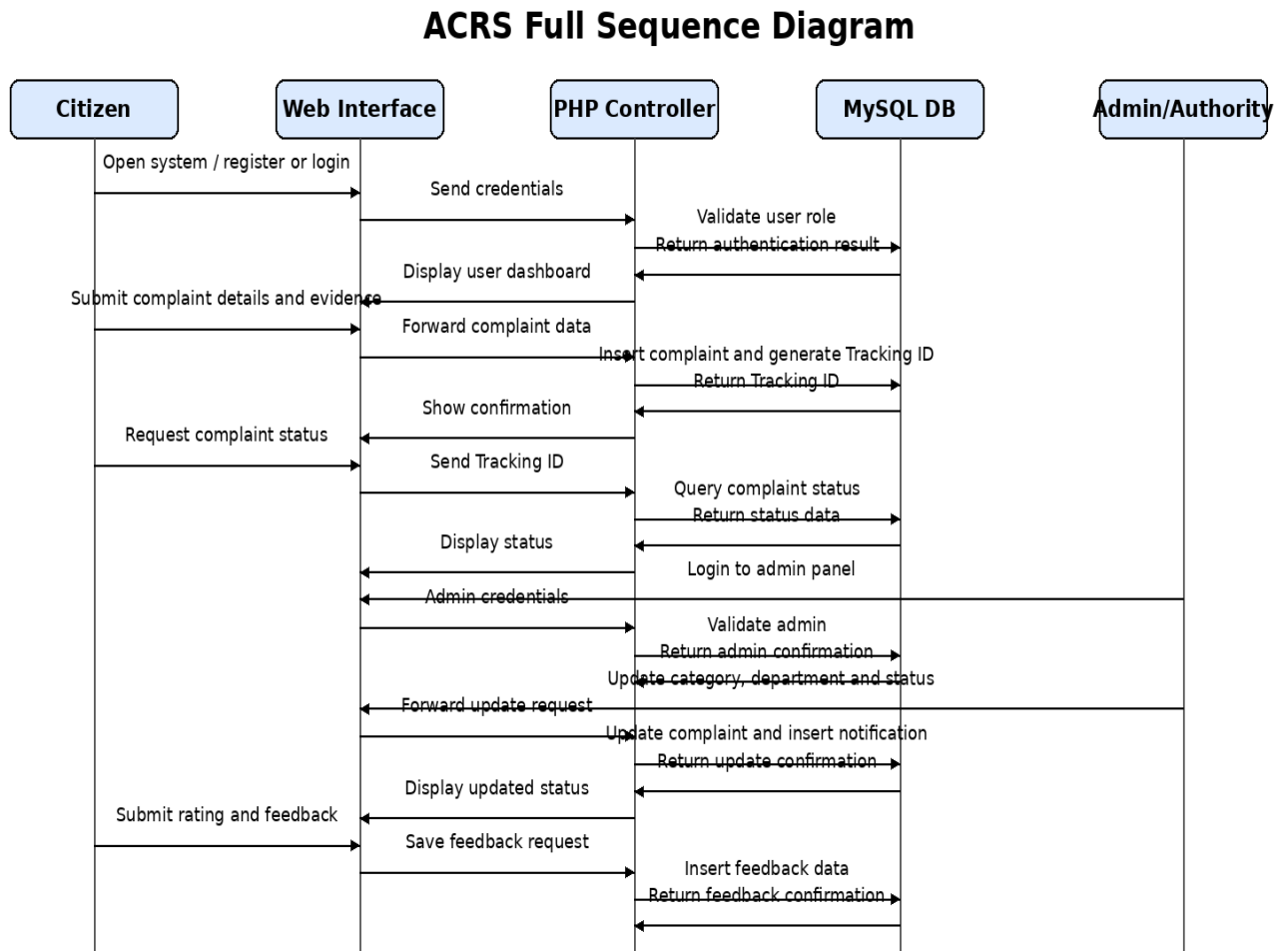


Figure 6: Full Sequence Diagram of ACRS

4.6 State Diagram

The State Diagram explains the life cycle of a complaint from drafting to submission, review, investigation, resolution, feedback, rejection, closure, and final state. This diagram is important because it shows how a complaint changes state during the entire process.

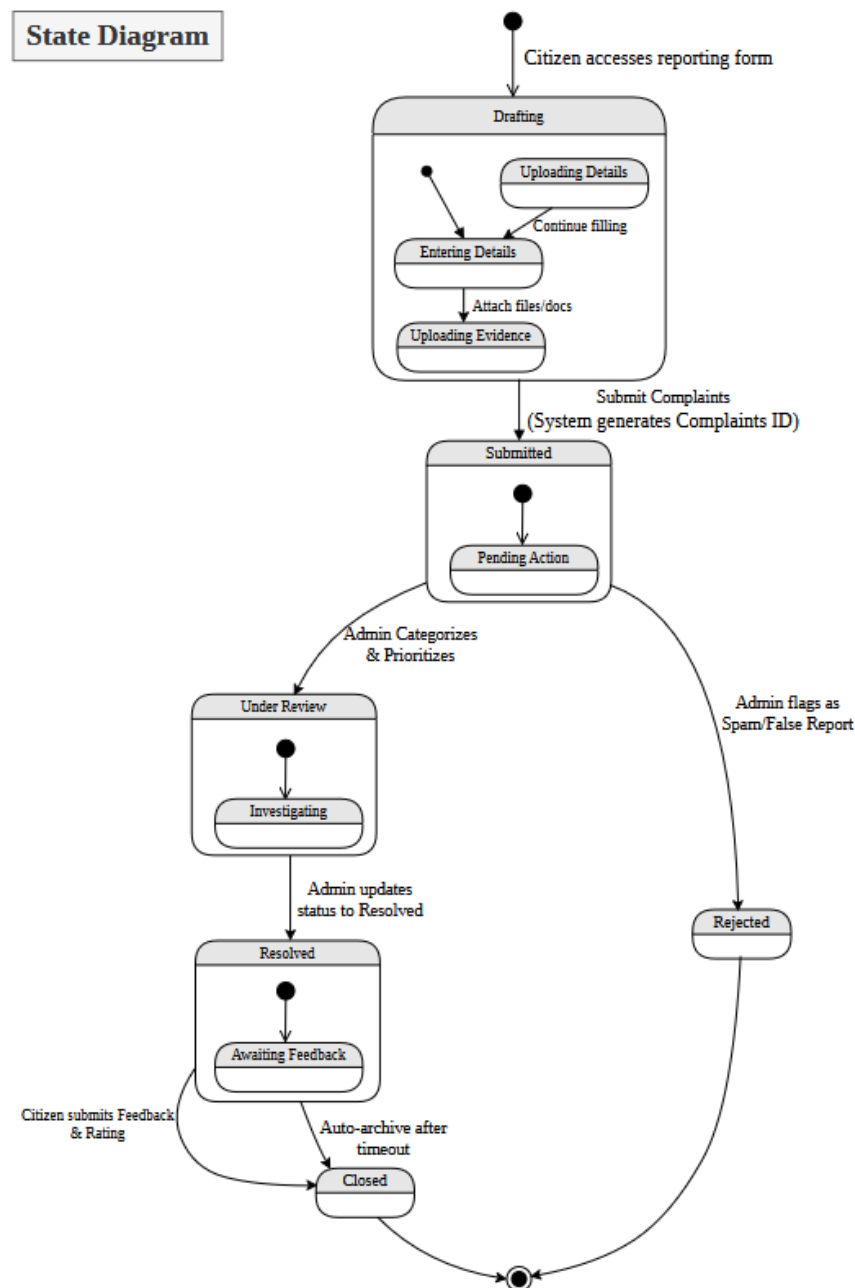


Figure 7: State Diagram of Complaint Processing

4.7 Database Design

The database design is based on four major tables: users, complaints, notifications, and feedback. The users table stores citizen and admin information. The complaints table stores report details, category, file path, status, anonymity flag, and tracking ID. The notifications table stores status alerts. The feedback table stores user ratings and comments.

Table Name	Important Fields	Purpose
users	id, name, email, password, role, created_at	Stores citizen and administrator accounts.
complaints	id, complaint_id, user_id, description, category, department, file_path, status, is_anonymous, created_at	Stores submitted corruption reports.
notifications	id, user_id, complaint_id, message, is_read, created_at	Stores status update messages for users.
feedback	id, complaint_id, rating, comment, submitted_at	Stores user rating and feedback after case resolution.

Chapter 5

System Implementation and User Interfaces

5.1 Technology Used

The project was implemented as a web-based system. HTML was used for page structure, CSS for styling, JavaScript for interactivity, PHP for backend processing, MySQL for database management, and XAMPP for local server operation.

5.2 Home Page

The homepage introduces the system with the message “Speak Up Against Corruption.” It contains navigation links such as Home, Login, Signup, and User Manual. The page encourages users to report misconduct safely.

5.3 Login Page

The login page allows registered users and administrators to enter email and password. The system checks the credentials and redirects users according to their role.

5.4 Signup Page

The signup page collects full name, email address, role, NID number, and password. After successful registration, the user receives a signup success message and can login.

5.5 User Manual

The user manual page explains how citizens can use the system. It provides instructions in Bengali for account creation, complaint submission, tracking, and admin updates.

5.6 Citizen Complaint Submission

The report page allows users to submit incident descriptions, choose category, upload evidence, and optionally submit anonymously. After submission, the system generates a unique tracking ID.

5.7 Admin Dashboard

The admin dashboard shows submitted complaints with tracking ID, reporter details, evidence link, department, current status, and management actions. Admins can update statuses and assign agencies.

5.8 Tracking and Feedback

The tracking page allows a citizen to enter the tracking ID and view the current complaint status. If the complaint is resolved, the user can submit rating and comments.



Figure 8: Home Page of ACRS

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

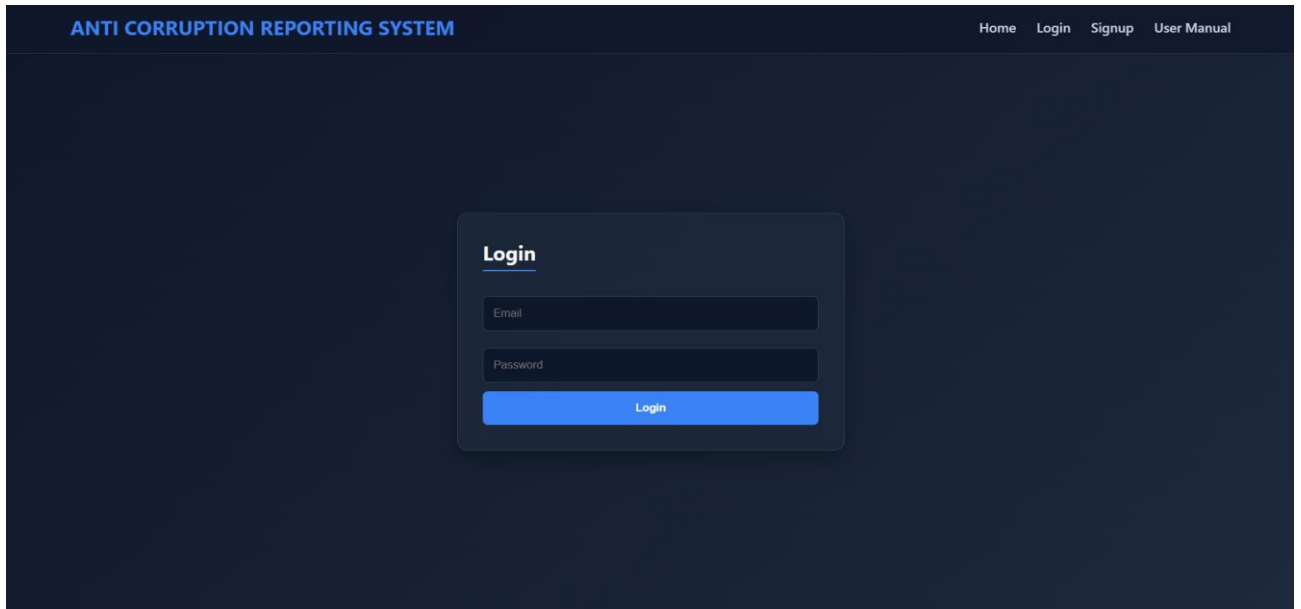
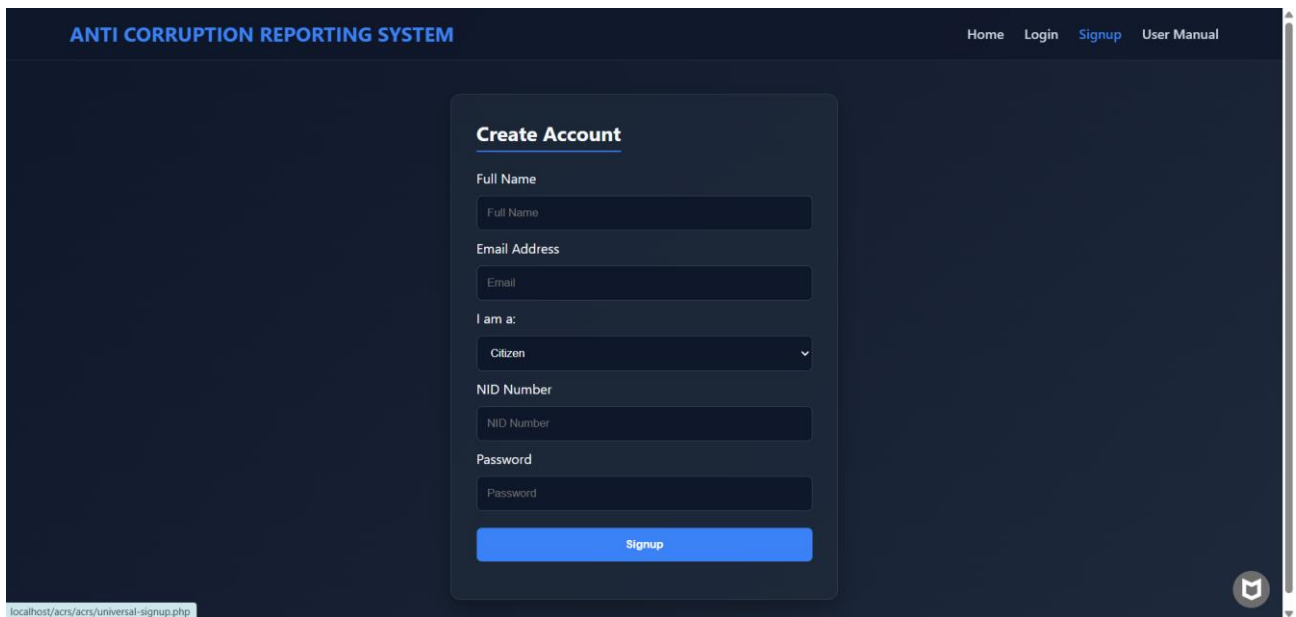


Figure 9: Login Page of ACRS

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.



The image shows a web browser window displaying the 'ANTI CORRUPTION REPORTING SYSTEM' (ACRS) signup page. The page has a dark blue header with the system name on the left and navigation links 'Home', 'Login', 'Signup', and 'User Manual' on the right. The 'Signup' link is highlighted in blue. In the center, there is a white 'Create Account' form. The form contains the following fields: 'Full Name' (text input), 'Email Address' (text input), 'I am a:' (dropdown menu with 'Citizen' selected), 'NID Number' (text input), and 'Password' (text input). A blue 'Signup' button is at the bottom of the form. The browser's address bar at the bottom left shows 'localhost/acrs/acrs/universal-signup.php'. A small circular logo is visible in the bottom right corner of the page.

ANTI CORRUPTION REPORTING SYSTEM

Home Login Signup User Manual

Create Account

Full Name

Email Address

I am a:

NID Number

Password

Signup

localhost/acrs/acrs/universal-signup.php

Figure 10: Signup Page of ACRS

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

The screenshot displays the 'ANTI CORRUPTION REPORTING SYSTEM' header with navigation links: Home, Login, Signup, and User Manual. The central 'Create Account' form includes fields for Full Name, Email Address, a dropdown for 'I am a:' (set to Citizen), NID Number, and Password. A blue 'Signup' button is at the bottom of the form. Below the button, a green message reads 'Signup Successfull' followed by a blue link 'Login here'.

Figure 11: Signup Success Message

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

The screenshot displays the 'ANTI CORRUPTION REPORTING SYSTEM' header with navigation links for Home, Login, Signup, and User Manual. The central 'Create Account' form includes input fields for Full Name (jubu), Email Address (jubu@gmail.com), a dropdown for 'I am a:' (Citizen), NID Number (45454545), and Password (masked with dots). A blue 'Signup' button is positioned below the password field. A green confirmation message, 'Signup Successfull! [Login here](#)', is visible at the bottom of the form.

Figure 12: User Manual Page in Bengali

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

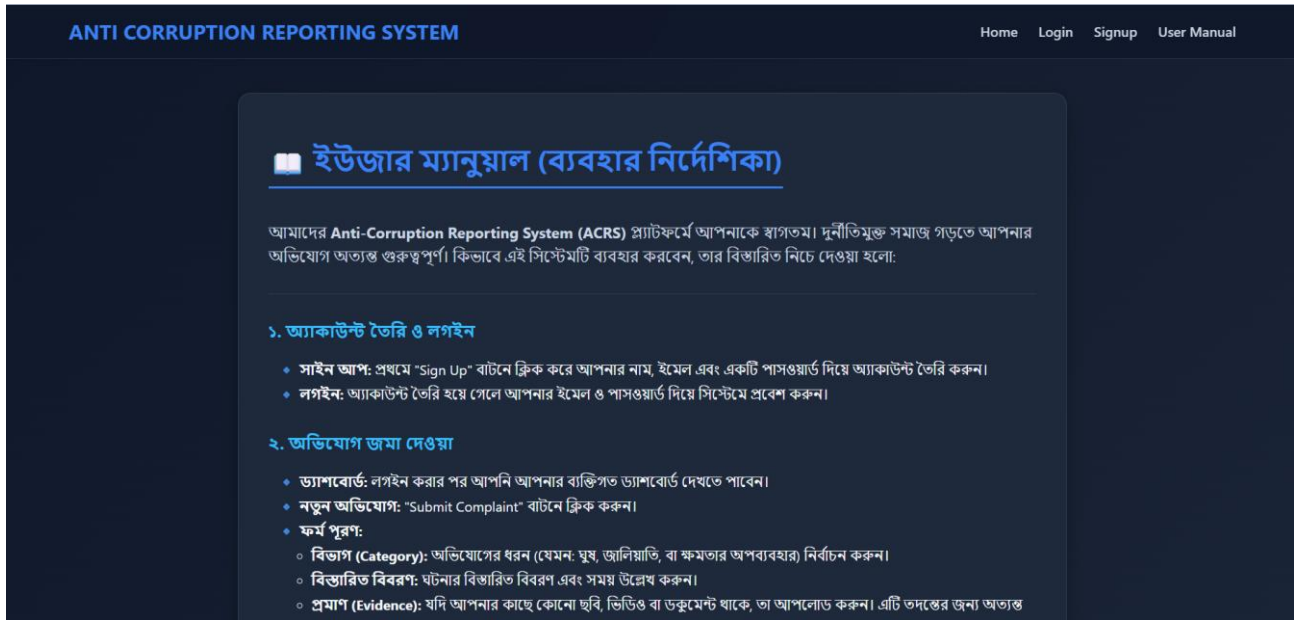


Figure 13: User Manual Status Tracking Instructions

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

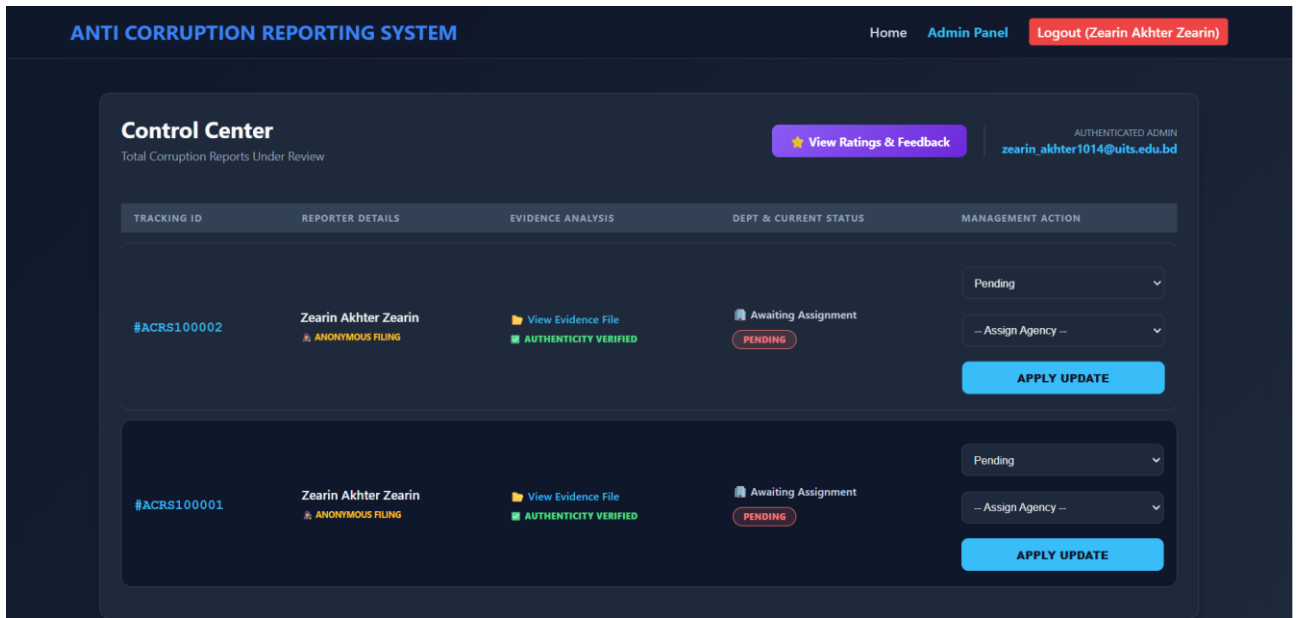


Figure 14: Submit Report Page

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

ANTI CORRUPTION REPORTING SYSTEM

Home Report My Complaints Track Logout (jubu)

Submit a Report

Incident Description

On 5th May 2026, a teacher demanded extra money from students in exchange for better grades in exams. Students who refused were treated unfairly and received lower marks.

Category

Education Sector Corruption

Attach Evidence (Optional)

Choose File Screenshot (1).png

Supported: JPG, PNG. Evidence is verified for authenticity.

Submit as Anonymous

Submit Report

Submitted! Tracking ID: ACRS100003

Figure 15: My Submitted Complaints Page

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

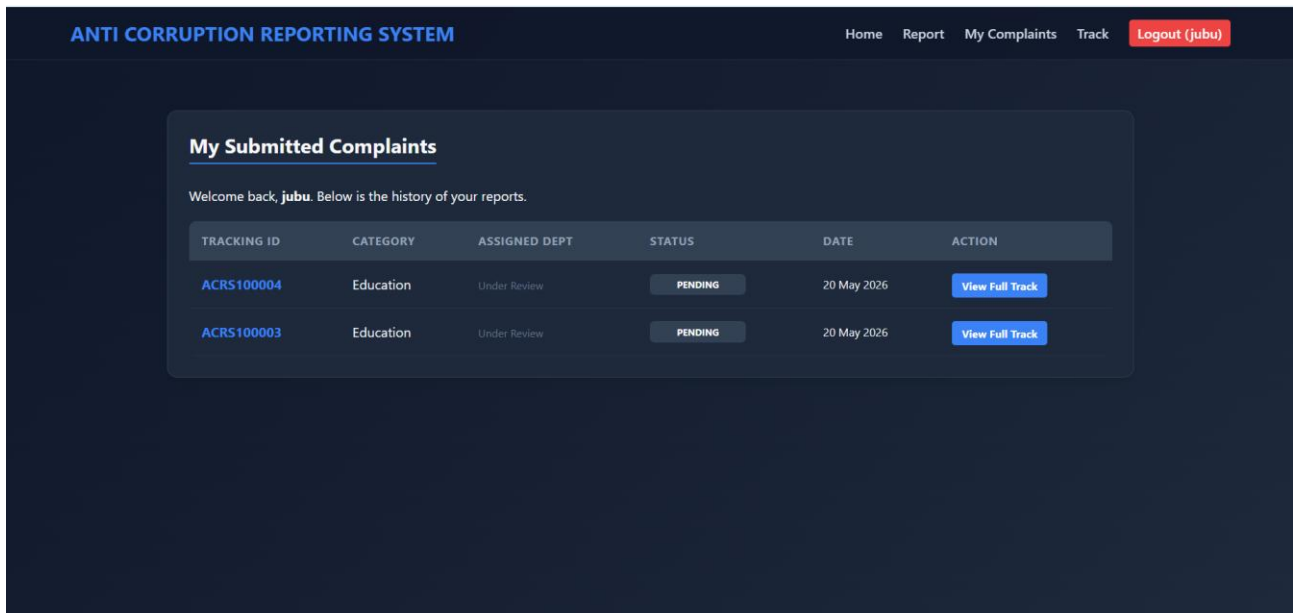


Figure 16: Admin Control Center with Reports

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

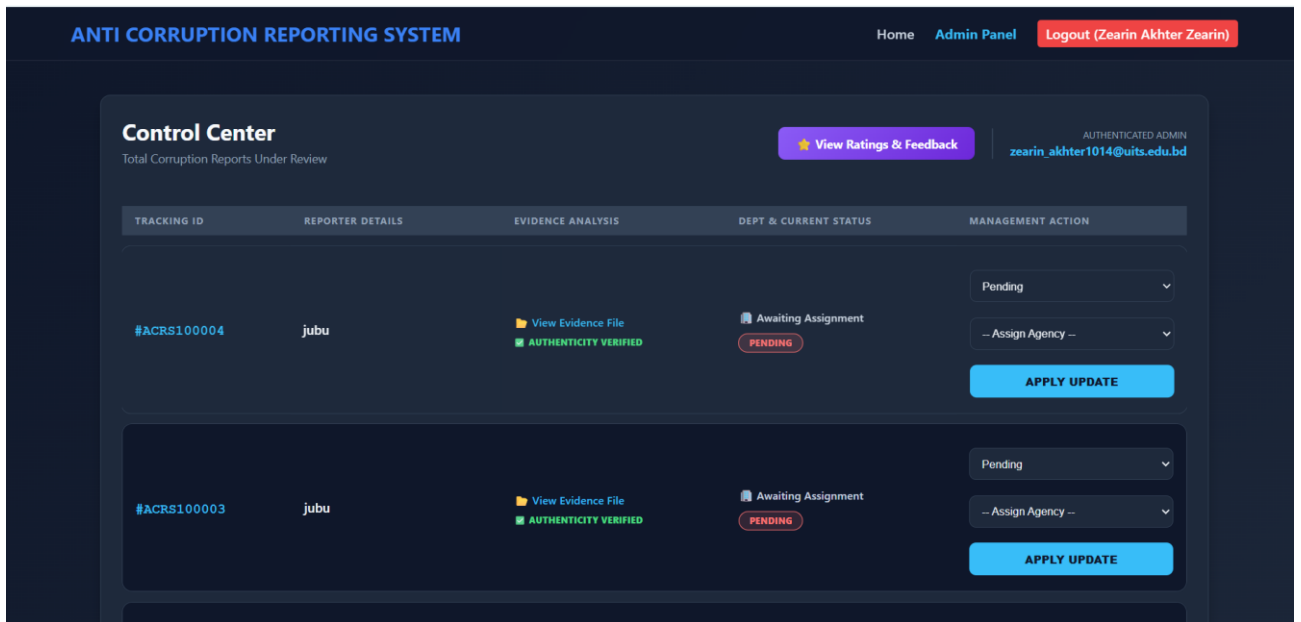


Figure 17: Admin Status Update Example

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

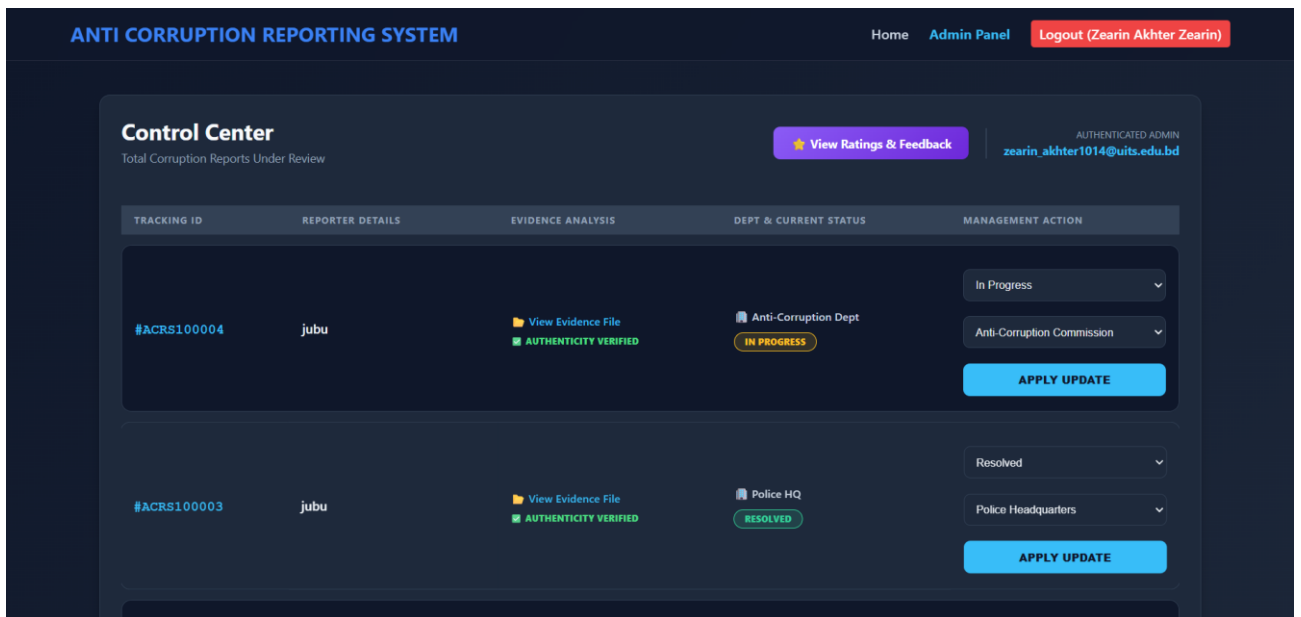


Figure 18: Updated Complaint List with Notification

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

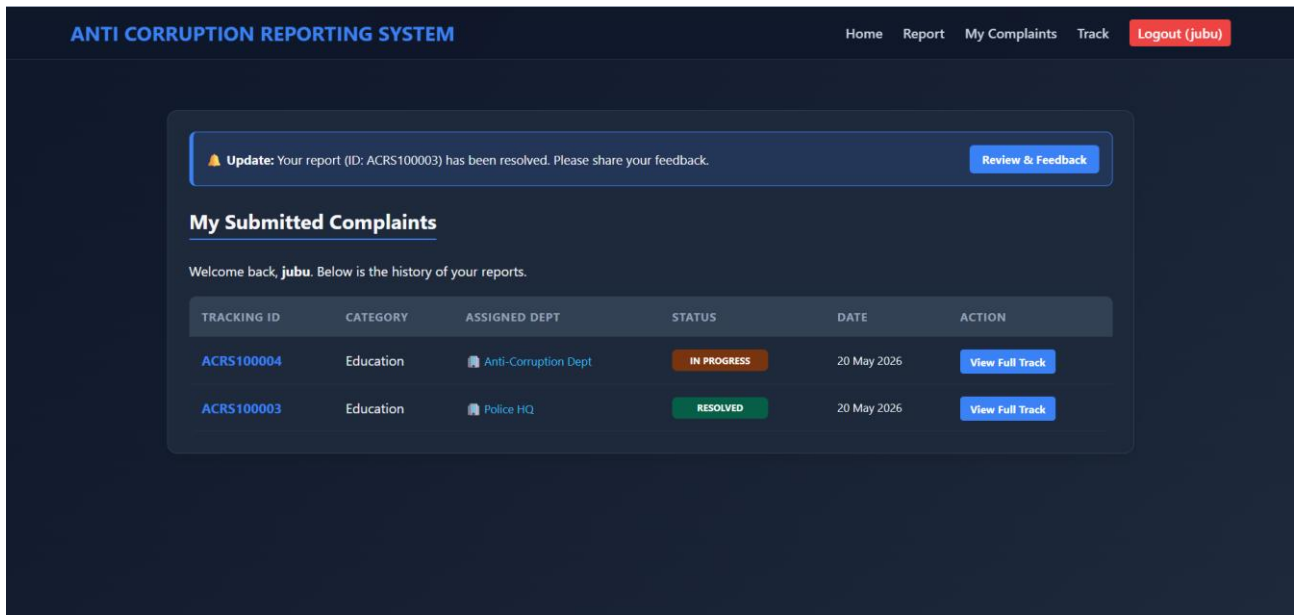


Figure 19: Track Complaint and Feedback Form

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

ANTI CORRUPTION REPORTING SYSTEM

[Home](#) [Report](#) [My Complaints](#) [Track](#) [Logout \(juba\)](#)

Enter your official Tracking ID (e.g., ACRS100001)

ACRS100003

Track

File Found: ACRS100003

Category: Education

Current Status: RESOLVED

🌟 Share Your Feedback

5 - Excellent

i'm happy with the service

Submit Feedback

Submitted On: 20 May 2026, 01:32 AM

Figure 20: Feedback Submitted View

This interface supports the main workflow of the ACRS project and demonstrates how the implemented system connects users, reports, tracking, and administrative actions.

5.9 Source Code File Structure

File Name	Purpose
index.php	Displays the homepage and main navigation.
universal-signup.php	Handles account creation for citizens and roles.
universal-login.php	Handles secure login for users and admins.
report.php	Allows citizens to submit complaints and evidence.
my-complaints.php	Displays complaint history for logged-in users.
track.php	Allows tracking by complaint ID and feedback submission.
admin.php	Displays admin dashboard for case management.
update_status.php	Processes admin status and department updates.
submit-feedback.php	Stores user feedback and rating.
db.php	Maintains database connection.
style.css	Controls the visual design of the system.

Chapter 6

System Testing

6.1 Testing Overview

Testing was performed to verify that the ACRS system works according to its functional requirements. The main focus was on authentication, complaint submission, evidence upload, tracking, admin updates, and feedback.

6.2 Testing Strategy

The project used black-box functional testing. Each major feature was tested from the user perspective by providing input values and checking expected output.

6.3 Testing Environment

Testing was performed on a local machine using XAMPP Apache server, MySQL database, PHP, and a Chrome web browser. The system was accessed through localhost.

6.4 Pass/Fail Criteria

A test case is considered passed if the system produces the expected result without error, stores or retrieves correct data, and displays proper feedback to the user.

Chapter 7

Discussion, Conclusion and Future Work

7.1 Discussion

The ACRS project successfully implements the major features required for a corruption reporting system. It provides a citizen-friendly reporting process, admin case management, tracking, evidence upload, and feedback collection. The system is simple but meaningful because it directly addresses transparency and accountability problems.

7.2 Limitations

The current version runs on a local server environment and is not deployed on a public domain. It also depends on user internet access and basic digital literacy. Advanced encryption, OTP verification, and official government integration are not yet implemented.

7.3 Future Scope

In the future, the system can be improved by adding mobile application support, SMS and email notification, AI-based false report detection, cloud hosting, advanced analytics dashboard, OTP verification, and multi-level authority approval.

7.4 Conclusion

The Anti-Corruption Reporting System is a practical web-based solution for reporting corruption securely and transparently. It helps citizens submit reports without fear, track complaint status, and provide feedback. It also helps administrators organize and update complaints efficiently. Overall, the system contributes to better accountability and public trust.

References

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

PHP Official Documentation.

MySQL Official Documentation.

XAMPP Apache Server Documentation.

Project Proposal: Anti-Corruption Reporting System, Spring 2026.

Software Requirements Specification for Anti-Corruption Reporting System, Version 1.0.

Core Implementation Code

Database Connection Code:

```
-- Select database
```

```
USE acrs;
```

```
-- 1. Users Table
```

```
CREATE TABLE IF NOT EXISTS users (  
    id INT NOT NULL AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    role ENUM('user', 'admin') NOT NULL DEFAULT 'user',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (id),  
    UNIQUE KEY unique_email (email)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
-- 2. Complaints Table
```

```
CREATE TABLE IF NOT EXISTS complaints (  
    id INT NOT NULL AUTO_INCREMENT,  
    complaint_id VARCHAR(20) DEFAULT NULL,  
    user_id INT NOT NULL,  
    description TEXT NOT NULL,  
    category VARCHAR(50) DEFAULT 'General',  
    department VARCHAR(100) DEFAULT NULL,
```

```

file_path VARCHAR(255) DEFAULT NULL,

status VARCHAR(20) DEFAULT 'Pending',

is_anonymous TINYINT(1) DEFAULT 0,

is_ai_generated TINYINT(1) DEFAULT 0,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

PRIMARY KEY (id),

INDEX (user_id),

FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 3. Notifications Table

```

CREATE TABLE IF NOT EXISTS notifications (

id INT NOT NULL AUTO_INCREMENT,

user_id INT NOT NULL,

complaint_id VARCHAR(20) NOT NULL,

message TEXT NOT NULL,

is_read TINYINT(1) DEFAULT 0,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

PRIMARY KEY (id),

INDEX (user_id),

FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

-- 4. Feedback Table

```

CREATE TABLE IF NOT EXISTS feedback (

```

```
id INT NOT NULL AUTO_INCREMENT,  
complaint_id VARCHAR(20) NOT NULL,  
rating TINYINT NOT NULL,  
comment TEXT,  
submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
PRIMARY KEY (id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

User Registration Code

Input Collection

```
$name = $_POST['name'] ?? '';
```

```
$email = $_POST['email'] ?? '';
```

```
$raw_password = $_POST['password'] ?? '';
```

Password validation

```
if (empty($raw_password)) {
```

```
    $msg = "Error: Password cannot be empty.";
```

```
}
```

Password Hashing

```
$hashed_password = password_hash($raw_password, PASSWORD_DEFAULT);
```

Duplicate Email Check

```
$checkEmail = $conn->prepare("SELECT email FROM users WHERE email = ?");
```

```
$checkEmail->bind_param("s", $email);
```

Login and Authentication Code

Session Initialization

```
if(session_status() === PHP_SESSION_NONE) session_start();
```

Input Collection

```
$email = trim($_POST['email']);
```

```
$pass = trim($_POST['password']);
```

Database Authentication Query

```
$stmt = $conn->prepare("SELECT * FROM users WHERE email = ? LIMIT 1");
```

```
$stmt->bind_param("s", $email);
```

```
$stmt->execute();
```

Role-Based Authentication

```
if (strtolower($user['role']) === 'admin') {
```

```
    header("Location: admin.php");
```

```
} else {
```

```
    header("Location: index.php");
```

```
}
```


Citizen Complaint Submission Code

Login Check

```
if (!isset($_SESSION['user'])) {  
    echo "<div class='center'><div class='card'><h3>Please <a href='universal-  
login.php'>Login</a> to submit a report.</h3></div></div>";  
    include 'footer.php';  
    exit();  
}
```

Complaint Input Collection

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    $desc = $_POST['desc'] ?? "";  
    $cat = $_POST['category'] ?? 'General';  
    $anon = isset($_POST['anon']) ? 1 : 0;  
    $uid = $_SESSION['user']['id'];  
}
```

Evidence file upload

Evidence Upload Logic

```
$filePath = "";

$sis_ai = 0;

if (!empty($_FILES['file']['name'])) {

    $uploadDir = "uploads/";

    if (!is_dir($uploadDir)) mkdir($uploadDir, 0777, true);

    $fileName = time() . "_" . basename($_FILES["file"]["name"]);

    $target = $uploadDir . $fileName;

    if (move_uploaded_file($_FILES["file"]["tmp_name"], $target)) {

        $filePath = $target;    }}
```

Complaint Tracking Code

Complaint Retrieval Query

```
$stmt = $conn->prepare("SELECT * FROM complaints WHERE user_id = ? ORDER BY id  
DESC");
```

```
$stmt->bind_param("i", $uid);
```

```
$stmt->execute();
```

```
$res = $stmt->get_result();
```

Notification System Query

```
$notif_query = $conn->prepare("SELECT * FROM notifications WHERE user_id = ? AND  
is_read = 0 ORDER BY created_at DESC");
```

Tracking Link

```
<a href="track.php?id=<?= $row['complaint_id'] ?>">
```

Admin Panel

```
<?php

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

// Check 1: Is a user logged in?

// Check 2: Is their role specifically 'admin'?

if (!isset($_SESSION['user']) || $_SESSION['user']['role'] !== 'admin') {
    // If not, send them away to the main login
    header("Location: login.php?error=unauthorized");
    exit();
}

?>
```

Case Management Code

Admin Authorization Check

```
$current_user = isset($_SESSION['user']['email']) ?  
trim(strtolower($_SESSION['user']['email'])) : "";  
  
if (!isset($_SESSION['user']) || !in_array($current_user, $admin_emails)) {  
    exit("Unauthorized Action");  
}
```

Case Status and Department Input

```
$id = isset($_POST['id']) ? intval($_POST['id']) : 0;  
$status = trim($_POST['status']) ?? 'Pending';  
$department = trim($_POST['department']) ?? '';
```

Case Update Query

```
$stmt = $conn->prepare("UPDATE complaints SET status = ?, department = ? WHERE id =  
?");  
$stmt->bind_param("ssi", $status, $department, $id);
```

Notification After Case Resolution

```
if ($status === 'Resolved') {  
  
    $info_stmt = $conn->prepare("SELECT user_id, complaint_id FROM complaints WHERE  
id = ?");  
  
    $info_stmt->bind_param("i", $id);  
  
    $info_stmt->execute();  
  
    $info_res = $info_stmt->get_result();  
  
    if ($info_row = $info_res->fetch_assoc()) {  
  
        $reporter_user_id = $info_row['user_id'];  
  
        $complaint_id = $info_row['complaint_id'];  
  
        $msg = "Your report (ID: $complaint_id) has been resolved. Please share your  
feedback.";  
  
        $notif_stmt = $conn->prepare("INSERT INTO notifications (user_id, complaint_id,  
message) VALUES (?, ?, ?)");  
  
        $notif_stmt->bind_param("iss", $reporter_user_id, $complaint_id, $msg);  
  
        $notif_stmt->execute();  
  
    }  
}
```

Status Update Code

```
$status = trim($_POST['status'] ?? 'Pending');  
  
$department = trim($_POST['department'] ?? '');  
  
$stmt = $conn->prepare("UPDATE complaints SET status = ?, department = ? WHERE id =  
?");  
  
$stmt->bind_param("ssi", $status, $department, $id);  
  
if ($stmt->execute()) {  
    header("Location: admin.php?msg=Success: Case updated to $status");  
    exit();  
}
```

Feedback and Rating Code

```
$rating = $_POST['rating'];  
  
$comment = $_POST['comment'];  
  
$complaint_id = $_POST['complaint_id'];  
  
$stmt = $conn->prepare("INSERT INTO feedback (complaint_id, rating, comment) VALUES  
(?, ?, ?)");  
  
$stmt->bind_param("sis", $complaint_id, $rating, $comment);  
  
if ($stmt->execute()) {  
    echo "Feedback submitted successfully."  
}
```

Feedback Display Code

```
$result = $conn->query("SELECT rating, comment FROM feedback WHERE  
complaint_id='$id'");  
  
while($row = $result->fetch_assoc()) {  
    echo $row['rating'];  
    echo $row['comment'];  
}
```


Logout and Session Management Code

Session Initialization and Authentication

```
session_start();  
  
if (!isset($_SESSION['user'])) {  
    header("Location: universal-login.php");  
    exit();  
}
```

User Session Storage After Login

```
$_SESSION['user'] = $user;
```

Logout Logic

```
session_start();  
  
session_unset();  
  
session_destroy();  
  
header("Location: universal-login.php");  
  
exit();
```